Sockets en Java: lo mínimo para la práctica 2

Documento de apoyo · Entrega voluntaria de la práctica 2 hasta el domingo 4 de octubre a las 23:59, sin nota

En clase hemos visto los sockets deprisa, y muchos de vosotros no los habíais visto antes. Aquí está lo que hace falta para escribir la práctica 2 y para saber qué estáis mirando cuando no funciona. Se lee en un cuarto de hora. Las preguntas del enunciado no se contestan aquí: son vuestras.

1. Qué es un socket

Un socket es un extremo de una conexión. Lo crea el sistema operativo, y el programa lo maneja casi igual que maneja un fichero: escribe bytes por un lado y lee bytes por el otro. Casi todo lo que sabéis hacer con un flujo (ingl. stream) sobre un fichero sirve tal cual sobre un socket. Lo que un socket añade, sobre todo, es la manera de abrirlo y unas cuantas maneras nuevas de fallar. Conviene saber desde el principio que muchas de esas maneras no producen ningún mensaje: se manifiestan como silencio, y eso desconcierta la primera vez.

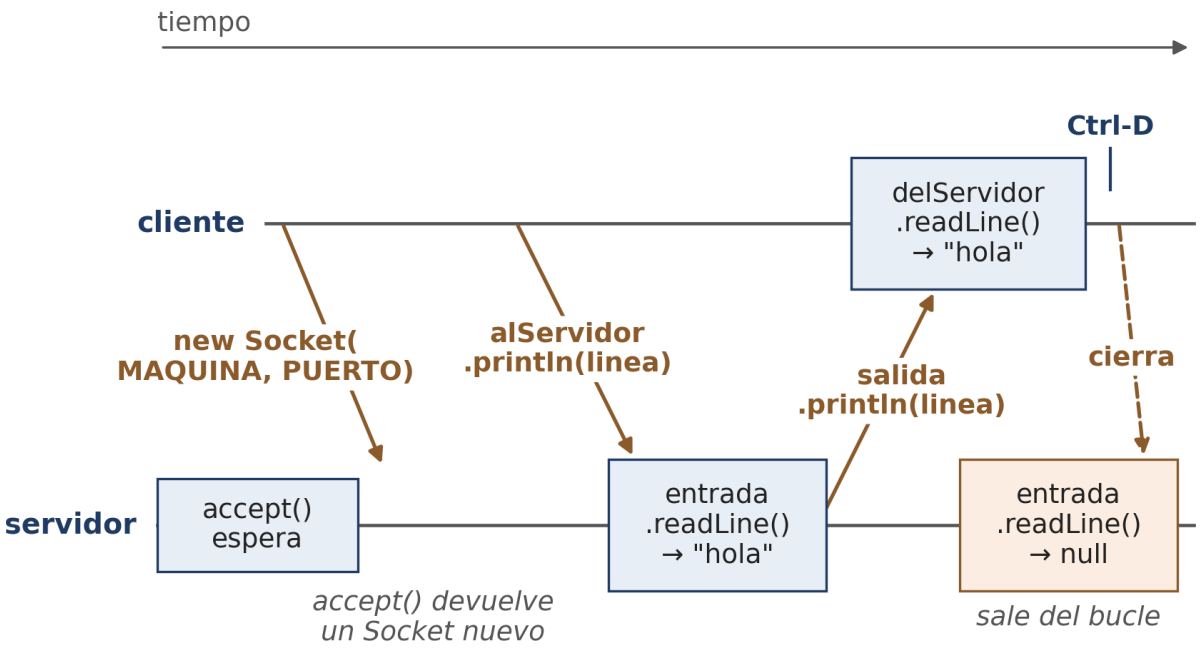
Una conexión queda identificada por cuatro datos: la dirección y el puerto del otro extremo, y la dirección y el puerto del propio. El servidor escucha en un puerto acordado de antemano, el 9999 en nuestra práctica, porque el cliente tiene que saber adónde llamar. El puerto del cliente se lo asigna su propio sistema operativo en el momento de conectar, y cambia de una ejecución a otra. Por eso varios clientes del mismo servidor pueden hablar todos con el puerto 9999 sin confundirse: lo que distingue una conexión de otra es el extremo del cliente.

2. Quién llama a qué

El servidor crea primero un ServerSocket. Ese objeto sólo sirve para esperar conexiones; para hablar con un cliente hace falta otra cosa. Su método accept() se queda bloqueado mientras no haya ninguna conexión pendiente. Cuando la hay, devuelve un Socket nuevo, distinto del ServerSocket, y ese Socket es por donde se habla con ese cliente concreto. El cliente, por su parte, crea directamente un Socket dando la dirección y el puerto del servidor.

En la práctica 2 el servidor llama a accept() una sola vez, de modo que tiene un único Socket por muchos clientes que lleguen. Un servidor que atienda a varios a la vez, que es lo que hace la práctica 3, tendrá un ServerSocket y un Socket por cada cliente.

Este es el intercambio completo de la práctica 2, con el tiempo avanzando de izquierda a derecha.



El eco monohilo: una conexión, una línea de ida y vuelta, y el final del flujo.

3. Los flujos del cliente, y cuáles van por la red

Aquí es donde casi todo el mundo se pierde. El cliente construye tres objetos de flujo, y además usa System.out, que ya viene construido. Sólo dos de los cuatro tocan la red. Los nombres son los del esqueleto del enunciado.

Objeto	Construido sobre	Lee de / escribe en
`teclado`	`System.in`	el teclado
`alServidor`	`socket.getOutputStream()`	la red
`delServidor`	`socket.getInputStream()`	la red
`System.out`	la salida estándar	la pantalla

En el servidor, los dos objetos que tocan la red se llaman entrada y salida. Son de las mismas clases que los del cliente y hacen lo mismo; sólo cambian los nombres.

Un PrintWriter no sabe adónde escribe. Escribe en el flujo que se le haya dado al construirlo. Si se le da un fichero, escribe en el fichero; si se le da un socket, escribe en la red. Es el mismo objeto en los dos casos. System.out es de la misma familia, pero ya viene construido sobre la salida estándar, y por eso nadie se acuerda de que es un flujo como los demás. De ahí que estas dos líneas del cliente, que se parecen tanto, hagan cosas distintas:

```
alServidor.println(linea);            // a la red
System.out.println("eco: " + eco);    // a la pantalla
```

Fijaos en que las dos variables tampoco son la misma. En linea está lo que ha tecleado el usuario y en eco lo que ha devuelto el servidor. Si imprimís linea en lugar de eco, el programa parecerá correcto haga lo que haga el servidor, incluso si no hay servidor.

4. El segundo argumento que hace que la línea salga

Al construir un PrintWriter sobre un socket hay que pasarle true como segundo argumento. Ese argumento activa el autovaciado (ingl. autoflush): cada println envía su línea en cuanto se escribe. Sin él, la línea se queda en un búfer dentro de vuestro propio programa y no llega a viajar, así que el otro extremo espera

una línea que existe pero que no ha salido de la máquina. El enunciado os pide quitar ese true a propósito y mirar lo que pasa.

```
PrintWriter alServidor = new PrintWriter(socket.getOutputStream(), true);
```

El autovaciado actúa además con println, printf y format, y no con print ni con write, de modo que print produce el mismo silencio por otro camino. Si escribís print en lugar de println, el programa se queda parado por partida doble: la línea no sale, y print tampoco escribe el fin de línea, que es justamente lo que el otro extremo está esperando para darse por servido.

5. Cuando el otro cierra, readLine() devuelve null

Un flujo se acaba. Cuando el cliente cierra su socket, la siguiente llamada a readLine() en el servidor no espera indefinidamente ni lanza una excepción. Devuelve null, que es como se dice en Java que el flujo ha terminado. El fin de fichero no viaja por el socket: lo que viaja es el cierre de la conexión, y readLine() lo traduce a null.

Vuestro bucle tiene que contar con eso y comprobarlo antes de usar la línea. Así es como lo hace el servidor:

```
String linea = entrada.readLine();
while (linea != null) {
    salida.println(linea);
    linea = entrada.readLine();
}
```

Si no lo comprobáis, el bucle no se entera de que la conversación ha terminado. Qué ocurre exactamente entonces es una de las preguntas del enunciado, y merece la pena ir a verlo en vuestra propia máquina.

6. Tropiezos de taller que no son parte del ejercicio

Esto es lo que suele costar la primera media hora de laboratorio y no tiene nada que ver con las preguntas del enunciado.

Lo que veis	**Qué hacer**
Nada más pegar el esqueleto, el entorno marca en rojo IOException, Socket o BufferedReader	Faltan las dos líneas de importación: `import java.io.*;` y `import java.net.*;`. En IntelliJ, Alt+Intro sobre el nombre en rojo (Opción+Intro en un Mac)
El servidor dice «Address already in use»	Hay otro servidor vuestro todavía vivo ocupando el puerto 9999, casi siempre de una prueba anterior. Paradlo con el cuadrado rojo de IntelliJ, con Ctrl+C en su terminal o, si no aparece, matando el proceso java
Escribís y no pasa nada	Estáis escribiendo en la pestaña del servidor. La entrada por teclado es la del cliente
El fin de fichero no hace nada, o Ctrl+C no detiene el programa	En la consola de IntelliJ, Ctrl+C no detiene nada: se usa el cuadrado rojo. El fin de fichero tiene su propia tecla, que va en el apartado 7
El entorno no os deja arrancar un segundo cliente, o para el primero	La configuración de ejecución admite una sola instancia. En IntelliJ: Modify options → Allow multiple instances
Las letras acentuadas vuelven como «?» en un terminal de Windows	Es la codificación de caracteres del terminal, no vuestro programa. No afecta a la práctica, y en la consola del entorno no ocurre

7. Cómo se termina un cliente

El cliente lee del teclado hasta el fin de fichero (ingl. end of file), y la tecla que lo produce depende de dónde lo estéis ejecutando.

Dónde	**Qué pulsar**
Terminal de Linux o de macOS	Ctrl+D
Terminal de Windows	Ctrl+Z al principio de una línea, y después Enter
Consola de IntelliJ IDEA	Ctrl+D (Cmd+D en macOS)
Si el programa se ha quedado esperando	El cuadrado rojo de IntelliJ, o Ctrl+C en un terminal

8. Cerrar el socket, y por qué a veces sobra

Socket implementa Closeable y, por tanto, AutoCloseable. Un socket declarado en la cabecera de un try se cierra solo al salir del bloque, salga por donde salga. Escribir después un close() explícito no hace daño, porque cerrar algo que ya está cerrado no tiene efecto, pero tampoco hace nada y despista a quien lea el código. Esta es la cabecera del cliente, con los cuatro recursos del esqueleto:

```
try (Socket socket = new Socket(MAQUINA, PUERTO);
     PrintWriter alServidor =
             new PrintWriter(socket.getOutputStream(), true);
     BufferedReader delServidor = new BufferedReader(
             new InputStreamReader(socket.getInputStream()));
     BufferedReader teclado = new BufferedReader(
             new InputStreamReader(System.in))) {
    // aquí va la conversación; no hace falta cerrar nada
}
```

Los recursos se cierran en orden inverso al de declaración. Conviene saber además que el socket y sus flujos están atados: cerrar cualquiera de los flujos cierra el socket, y cerrar el socket deja inservibles los flujos. Cerrar dos veces no tiene efecto, así que los cuatro pueden ir en la misma cabecera sin estorbarse. Lo que sí hay que evitar es meter en la cabecera de un try el flujo de un socket que queráis seguir usando cuando el bloque termine, porque al cerrarse el flujo se cierra también el socket.

El enunciado está en Moodle, con los dos esqueletos y las preguntas. La entrega es voluntaria y no lleva nota: se corrige y se os devuelve un comentario. Si algo de esto no encaja con lo que veis en vuestra máquina, traedlo a clase el martes 29.